

VIDEO GAME DATA ENHANCEMENT USING GOOGLE AI STUDIO API

Objective

The objective of this project is to enrich a video game dataset using Google AI Studio API to automatically generate:

- genre classification
- short descriptions
- player mode metadata

Tools & Technologies

- Python
- Pandas
- Google AI Studio API
- Gemini Flash Model
- JSON based Prompt Engineering

```
Game Thumbnail.csv X
task 2 > Game Thumbnail.csv > data
1 game_title,image_url
2 Street Fighter 6,https://images.igdb.com/igdb/image/upload/t_cover_big/co5vst.jpg
3 Hunt: Showdown 1896,https://images.igdb.com/igdb/image/upload/t_cover_big/co8o7i.jpg
4 Wuthering Waves,https://images.igdb.com/igdb/image/upload/t_cover_big/co92hl.jpg
5 Arma Reforger,https://images.igdb.com/igdb/image/upload/t_cover_big/co4rk4.jpg
```

- The original dataset only contained:
game titles & image URLs

Additional metadata was generated
through prompt based AI inference

- **The workflow combines:**

- prompt engineering
- API integration
- structured response parsing
- automated CSV generation



PROMPT ENGINEERING STRATEGY

The prompt was carefully structured to constrain output formatting and improve response consistency.

Specific rules were added to:

- reduce hallucinated formatting
- simplify automated parsing
- improve output reliability
- standardize generated metadata

The prompt also enforced:

- single word genre classification
- descriptions under 30 words
- restricted player mode categories
- valid JSON only responses

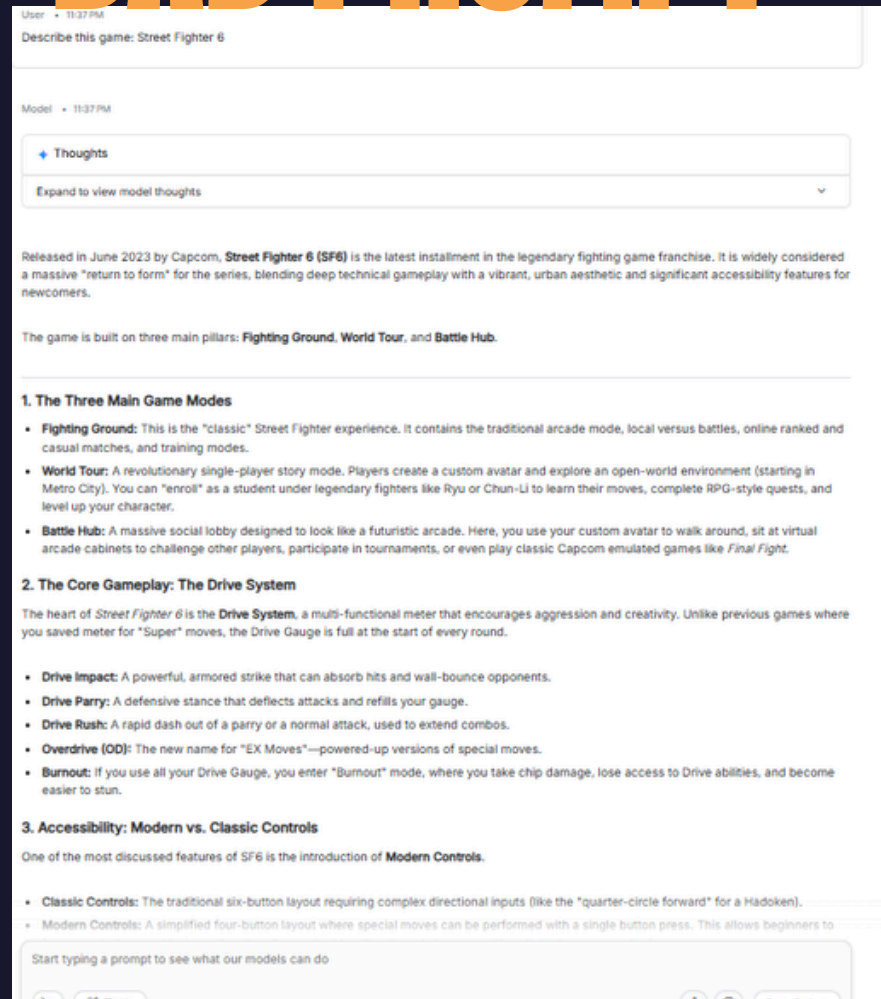
Based on the processing results, structured prompting significantly reduced formatting inconsistencies during large scale dataset generation

Constraining the response format reduced parsing failures during automated dataset generation

The prompt design prioritized consistency and machine readable outputs instead of conversational responses

```
def build_prompt(title):  
  
    return f"""  
You are a video game metadata assistant.  
  
Game Title:  
{title}  
  
Task 1 - Genre Classification  
Rules:  
- Return ONLY one genre word  
- Example:  
Shooter  
RPG  
Strategy  
  
Task 2 - Short Description  
Rules:  
- Maximum 30 words  
- Concise and engaging  
- No extra explanation  
  
Task 3 - Player Mode  
Rules:  
- Return ONLY one:  
Singleplayer  
Multiplayer  
Both  
Return ONLY valid JSON format.  
  
Example:  
{{  
    "genre": "Shooter",  
    "short_description": "Fast-paced multiplayer shooter with  
tactical combat and competitive gameplay.",  
    "player_mode": "Multiplayer"  
}}  
"""
```

BAD PROMPT



Problems:

- inconsistent formatting
- unpredictable responses
- difficult automated parsing
- lower processing reliability

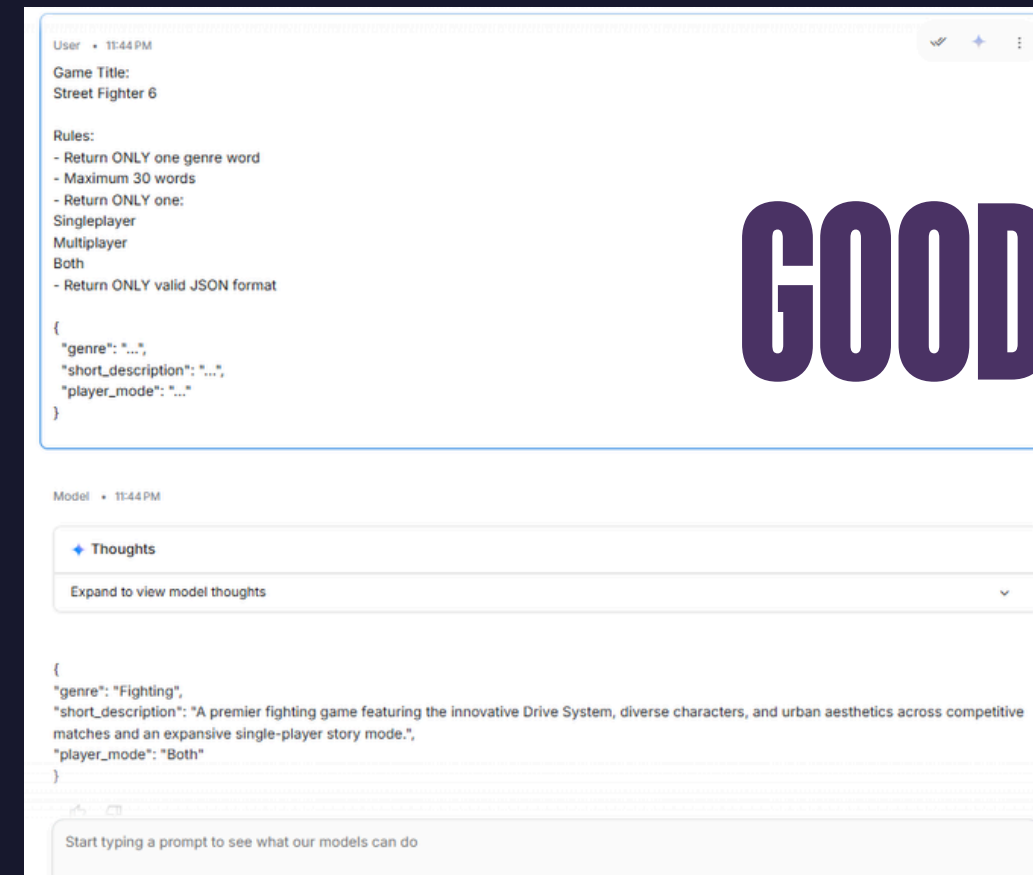
The initial prompt was too broad and lacked formatting constraints

As a result, the model generated:

- inconsistent response structures
- unpredictable formatting
- long unstructured explanations

VS

GOOD PROMPT



Explicit formatting instructions were added to constrain how the model generated responses

Structured JSON outputs improved:

- consistency
- parsing reliability
- scalability
- automation workflows

Benefits:

- structured output
- scalable processing
- easier automation
- more reliable parsing

Adding explicit formatting constraints significantly improved output quality during large scale dataset processing

CHALLENGES & SOLUTIONS

```
Untitled

except (
    ResourceExhausted,
    TooManyRequests,
    InternalServerError,
    ServiceUnavailable,
    Aborted
) as e:

    print(f"Retry {attempt+1} API limit or temporary
service issue: {e}")

    time.sleep(delay)
    delay = min(delay * 2, 120)
```

During large scale processing, several API related issues occurred, particularly:

- rate limiting
- temporary service failures
- inconsistent responses
- JSON parsing errors

Large datasets increase the likelihood of interrupted processing, especially during repeated API requests

RETRY HANDLING IMPLEMENTATION

To improve processing stability, retry handling and delay mechanisms were implemented inside the workflow

Fallback values were also used to prevent execution failure when invalid responses occurred

Exponential backoff retry logic helped stabilize execution during temporary API failures and rate limit interruptions

 enhanced_games_backup.csv

Incremental CSV backups were used to reduce data loss risk during interrupted execution

CONCLUSION & REFLECTIONS



KEY LEARNINGS

- prompt structure strongly affects output quality
- structured JSON responses simplify automation workflows
- constrained outputs improve consistency across large datasets
- retry handling improves API reliability during longrunning execution

FUTURE IMPROVEMENTS

- multi label genre classification
- image based game understanding
- confidence scoring for generated outputs
- asynchronous processing
- for faster execution

The project demonstrates how generative AI can automate metadata enrichment workflows efficiently, while highlighting the importance of:

- prompt engineering
- response validation
- scalable API handling